Digital Information Processing Lab 2025

Class 4: Filters

Group 14 : Snoopy

06.07.2025

Name	Mat. Nr.	Email
Yash Khiste	254751	yash.khiste@st.ovgu.de
Saiful Islam	253574	saiful1.islam@st.ovgu.de
Supritha Rajashekaraiah	244941	supritha.rajashekaraiah@ovgu.de

Class 4: Filters

Task: Filters

Get Acquainted:

- Familiarize yourself with the various filters and their parameters.

Choose Filters for ECG Signal:

- Select three filters to remove the three types of errors in the ECG signal *ecg_signal_all*.
- Explain which filter is suitable for each type of error and why.
- Set the filter parameters and demonstrate the before and after effects.
- Consider the order in which the filters should be applied.

Visualize the Filtering Process:

- Using the provided plot example, illustrate the filtering process of the ECG signal.
- Write a summary about the process

Introduction: Parts of the ECG

The electrocardiogram (ECG) is a recording of the electrical activity of the heart. It consists of various components, each representing specific electrical activities of the heart:

- P wave: Shows atrial depolarization (spread of excitation in the atria).
- QRS complex: Shows ventricular depolarization (spread of excitation in the ventricles).
- T wave: Shows ventricular repolarization (recovery of excitation in the ventricles).
- U wave (if present): May represent late repolarization of the Purkinje fibers or other parts of the heart.

Here is an example of a clean ECG signal:

ECG Generator

ECG Generator

[ ] ↳ 1 cell hidden

Introduction of Error Types

[ ] ↳ 9 cells hidden

Filters

Here are some implementations of filters:

High-pass filter

[ ] ↳ 3 cells hidden

Notch Filter

[ ] ↳ 4 cells hidden

Wavelet-Denoising

Wavelet-Denoising can be used to remove high-frequency noise such as Gaussian noise while preserving important features of the signal.

[ ] ↳ 3 cells hidden

Median Filter

A median filter can be used to remove impulse noise while preserving the edges of the signal.

[ ] ↳ 3 cells hidden

Moving Average Filter

A moving average filter can be used to smooth high-frequency noise by averaging over a window.

[ ] ↳ 3 cells hidden

Fourier Transform Filter

A Fourier transform filter can be used to smooth high-frequency noise by transforming the signal into the frequency domain, removing unwanted frequencies, and transforming the signal back into the time domain.

[ ] ↳ 3 cells hidden

Savitzky-Golay Filter

A Savitzky-Golay filter can be used to smooth high-frequency noise while preserving important features of the signal. This filter is particularly good at preserving the signal integrity as it fits a polynomial to the data points and smooths them accordingly.

[ ] ↳ 3 cells hidden

Adaptive Filtering with LMS

Adaptive filters continuously adjust to the characteristics of the input signal. The Least Mean Squares (LMS) algorithm is a commonly used adaptive filter, specifically designed for situations where the signal characteristics vary.

[ ] ↳ 3 cells hidden

Combination of Filters (Plot Example)

```
filtered_1 = apply_highpass_filter(ecg_signal_all, cutoff=0.5, fs=1.5*sampling_
filtered_1 = apply_notch_filter(filtered_1, cutoff=50, fs=sampling_rate, qualit
x=filtered_1
filtered_1 = apply_savgol_filter(filtered_1, window_length=31,polyorder=7)
final=filtered_1
```

```
filtered_2 = apply_highpass_filter(ecg_signal_all, cutoff=0.5, fs=sampling_rate
filtered_2 = apply_savgol_filter(filtered_2, window_length=11, polyorder=3)
filtered_2 = apply_wavelet_denoising(filtered_2, wavelet='db4', level=2)
```

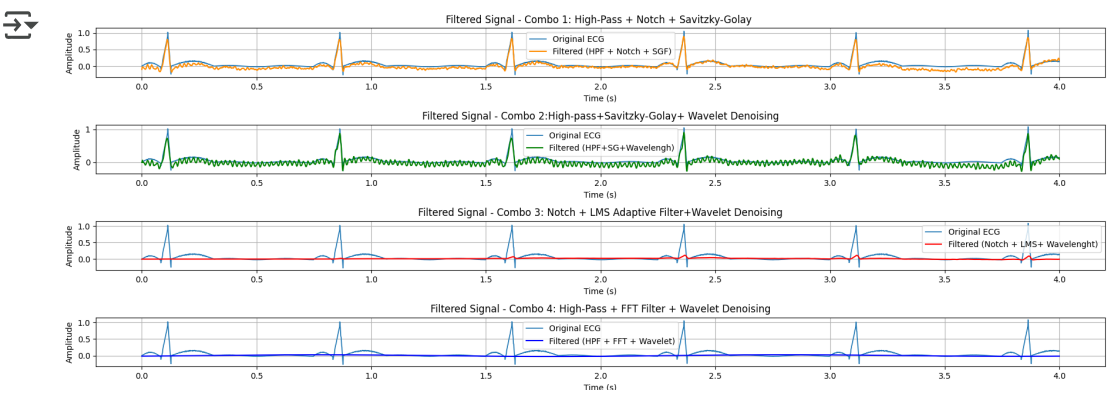
```
filtered_3 = apply_notch_filter(ecg_signal_all, cutoff=50, fs=sampling_rate, qu
filtered_3 = lms_filter(filtered_3, mu, order)
x = filtered_3
filtered_3 = apply_wavelet_denoising(filtered_3, wavelet='db4', level=1)
final = filtered_3
```

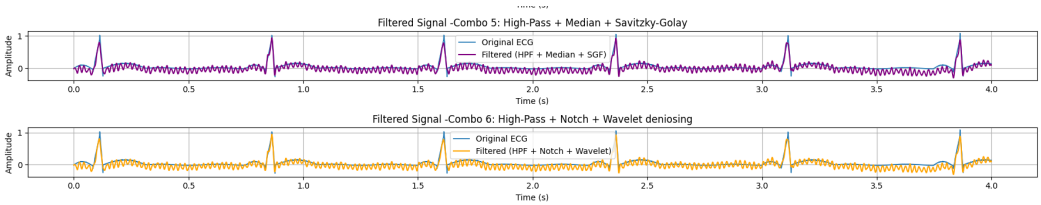
```
filtered_4 = apply_highpass_filter(ecg_signal_all, cutoff=0.5, fs=sampling_rate
filtered_4 = apply_fourier_filter(data=filtered_4,cutoff=0.5)
filtered_4 = apply_wavelet_denoising(filtered_4, wavelet='db4', level=1)
final_4 = filtered_4
```

```
filtered_5 = apply_highpass_filter(ecg_signal_all, cutoff=0.5, fs=sampling_rate
filtered_5 = apply_median_filter(filtered_5, kernel_size=5)
filtered_5 = apply_savgol_filter(filtered_5, window_length=11, polyorder=3)
final 5 = filtered 5
```

```
      _             _

filtered_6 = apply_highpass_filter(ecg_signal_all, cutoff=0.5, fs=sampling_rate
filtered_6 = apply_median_filter(filtered_6, kernel_size=5)
filtered_6 = apply_savgol_filter(filtered_6, window_length=11, polyorder=3)
final_6 = filtered_6



filtered_7 = apply_highpass_filter(ecg_signal_all, cutoff=0.5, fs=1.5 * samplir
filtered_7 = apply_notch_filter(filtered_7, cutoff=50, fs=sampling_rate, qualit
filtered_7 = apply_wavelet_denoising(filtered_7, wavelet='db4', level=1)
final = filtered_7


import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error

# --- Plotting Filtered vs Original ECG ---

plt.figure(figsize=(18, 10))

plt.subplot(6, 1, 1)
plt.plot(t, ecg_signal, label='Original ECG', linewidth=1.2)
plt.plot(t, filtered_1, label='Filtered (HPF + Notch + SGF)', color='darkorange
plt.title('Filtered Signal - Combo 1: High-Pass + Notch + Savitzky-Golay ')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)

plt.subplot(6, 1, 2)
plt.plot(t, ecg_signal, label='Original ECG', linewidth=1.2)
plt.plot(t, filtered_2, label='Filtered (HPF+SG+Wavelengh)', color='green')
plt.title('Filtered Signal - Combo 2:High-pass+Savitzky-Golay+ Wavelet Denoisir
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)

plt.subplot(6, 1, 3)
plt.plot(t, ecg_signal, label='Original ECG', linewidth=1.2)
plt.plot(t, filtered_3, label='Filtered (Notch + LMS+ Wavelenght)', color='red'
plt.title('Filtered Signal - Combo 3: Notch + LMS Adaptive Filter+Wavelet Denoi
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)


plt.subplot(6, 1, 4)
plt.plot(t, ecg_signal, label='Original ECG', linewidth=1.2)
plt.plot(t, filtered_4, label='Filtered (HPF + FFT + Wavelet)', color='blue')
plt.title('Filtered Signal - Combo 4: High-Pass + FFT Filter + Wavelet Denoisir
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)



plt.subplot(6, 1, 5)
plt.plot(t, ecg_signal, label='Original ECG', linewidth=1.2)
plt.plot(t, filtered_5, label='Filtered (HPF + Median + SGF)', color='purple')
plt.title('Filtered Signal -Combo 5: High-Pass + Median + Savitzky-Golay')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.legend()
```

```
plt.legend()
plt.grid(True)

plt.subplot(6, 1, 6)
plt.plot(t, ecg_signal, label='Original ECG', linewidth=1.2)
plt.plot(t, filtered_6, label='Filtered (HPF + Notch + Wavelet)', color='orange
plt.title('Filtered Signal –Combo 6: High–Pass + Notch + Wavelet deniosing')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Calculating MSE for Each Filter Combination ---

mse_1 = mean_squared_error(ecg_signal, filtered_1)
mse_2 = mean_squared_error(ecg_signal, filtered_2)
mse_3 = mean_squared_error(ecg_signal, filtered_3)
mse_4 = mean_squared_error(ecg_signal, filtered_4)
mse_5 = mean_squared_error(ecg_signal, filtered_5)
mse_6 = mean_squared_error(ecg_signal, filtered_6)
# === Percent Root Difference (PRD) ===
def prd(original, denoised):
    numerator = np.linalg.norm(original - denoised)
    denominator = np.linalg.norm(original)
    return (numerator / denominator) * 100

prd_1 = prd(ecg_signal, filtered_1)
prd_2 = prd(ecg_signal, filtered_2)
prd_3 = prd(ecg_signal, filtered_3)
prd_4 = prd(ecg_signal, filtered_4)
prd_5 = prd(ecg_signal, filtered_5)

prd_6 = prd(ecg_signal, filtered_6)
# === Output Results ===
print("Mean Squared Error (MSE) and Percent Root Difference (PRD)")
print(f"Combo 1 (High–Pass + Notch + Savitzky–Golay):")
print(f"   MSE: {mse_1:.5f}    PRD: {prd_1:.2f}%")

print(f"Combo 2 (High–pass + Savitzky–Golay + Wavelet Denoising):")
print(f"   MSE: {mse_2:.5f}    PRD: {prd_2:.2f}%")

print(f"Combo 3 (Notch + LMS Adaptive Filter + Wavelet Denoising):")
print(f"   MSE: {mse_3:.5f}    PRD: {prd_3:.2f}%")


print(f"Combo 4 (High–Pass + FFT Filter + Wavelet Denoising):")
print(f"   MSE: {mse_4:.5f}    PRD: {prd_4:.2f}%")

print(f"Combo 5 (High–Pass + Median + Savitzky–Golay):")
print(f"   MSE: {mse_5:.5f}    PRD: {prd_5:.2f}%")
print(f"Combo 6 (High–Pass + Notch + Wavelet deniosing):")
print(f"   MSE: {mse_6:.5f}    PRD: {prd_6:.2f}%")
```

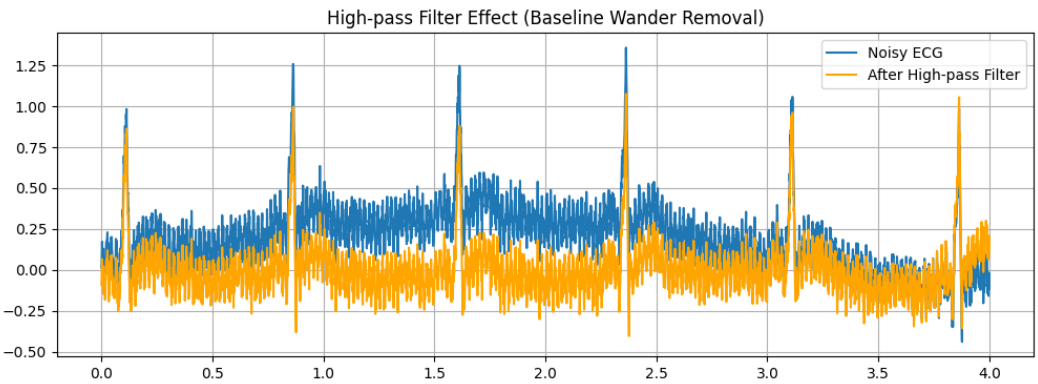




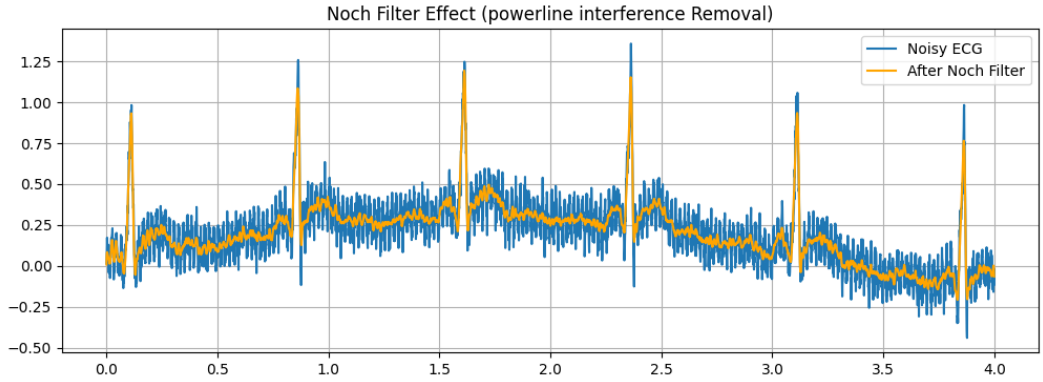

```
Mean Squared Error (MSE) and Percent Root Difference (PRD)
Combo 1 (High-Pass + Notch + Savitzky-Golay):
   MSE: 0.00529    PRD: 49.39%
Combo 2 (High-pass + Savitzky-Golay + Wavelet Denoising):
   MSE: 0.00763    PRD: 59.31%
Combo 3 (Notch + LMS Adaptive Filter + Wavelet Denoising):
   MSE: 0.01855    PRD: 92.49%
Combo 4 (High-Pass + FFT Filter + Wavelet Denoising):
   MSE: 0.02207    PRD: 100.88%
Combo 5 (High-Pass + Median + Savitzky-Golay):
   MSE: 0.00895    PRD: 64.23%
Combo 6 (High-Pass + Notch + Wavelet deniosing):
   MSE: 0.00895    PRD: 64.23%
```

Before and after effects of the filter individually

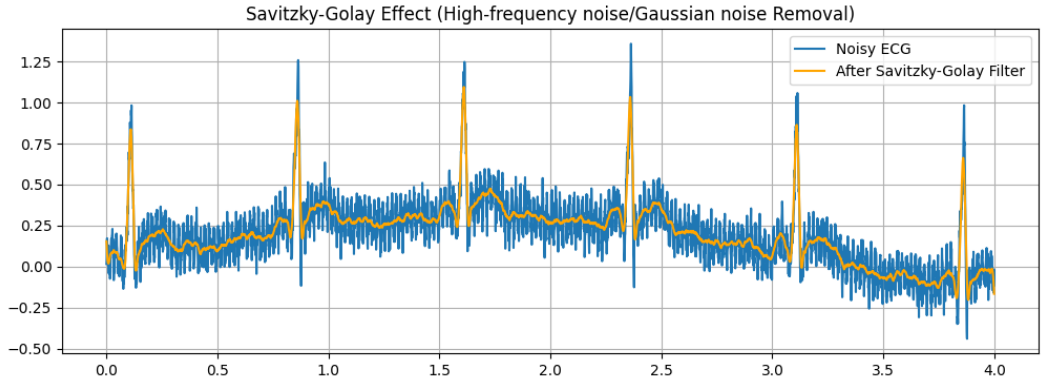
```
# High-pass only
hp_filtered = apply_highpass_filter(ecg_signal_all, cutoff=0.5, fs=1000, order=
plt.figure(figsize=(12,4))
plt.plot(t, ecg_signal_all, label="Noisy ECG")
plt.plot(t, hp_filtered, label="After High-pass Filter", color='orange')
plt.title("High-pass Filter Effect (Baseline Wander Removal)")
plt.legend()
plt.grid(True)
plt.show()
```



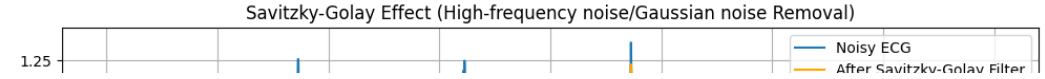
```
# Noch Filter only
hp_filtered = apply_notch_filter(filter1, cutoff=50, fs=1000, quality_factor=30
plt.figure(figsize=(12,4))
plt.plot(t, ecg_signal_all, label="Noisy ECG")
plt.plot(t, hp_filtered, label="After Noch Filter", color='orange')
plt.title("Noch Filter Effect (powerline interference Removal)")
plt.legend()
plt.grid(True)
plt.show()
```

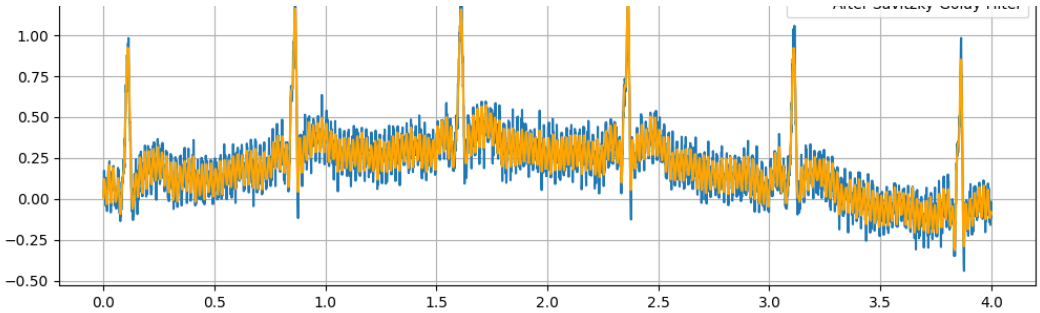


```
# Savitzky-Golay Filter only
hp_filtered = savgol_filter(ecg_signal_all, window_length=35, polyorder=3)
plt.figure(figsize=(12,4))
plt.plot(t, ecg_signal_all, label="Noisy ECG")
plt.plot(t, hp_filtered, label="After Savitzky-Golay Filter", color='orange')
plt.title("Savitzky-Golay Effect (High-frequency noise/Gaussian noise Removal)'
plt.legend()
plt.grid(True)
plt.show()
```

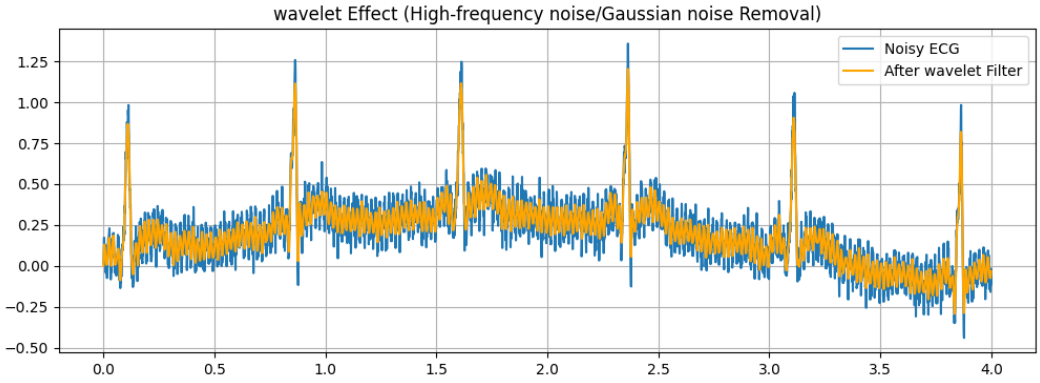



```
# Savitzky-Golay Filter only
hp_filtered = savgol_filter(ecg_signal_all, window_length=31, polyorder=7)
plt.figure(figsize=(12,4))
plt.plot(t, ecg_signal_all, label="Noisy ECG")
plt.plot(t, hp_filtered, label="After Savitzky-Golay Filter", color='orange')
plt.title("Savitzky-Golay Effect (High-frequency noise/Gaussian noise Removal)'
plt.legend()
plt.grid(True)
plt.show()
```

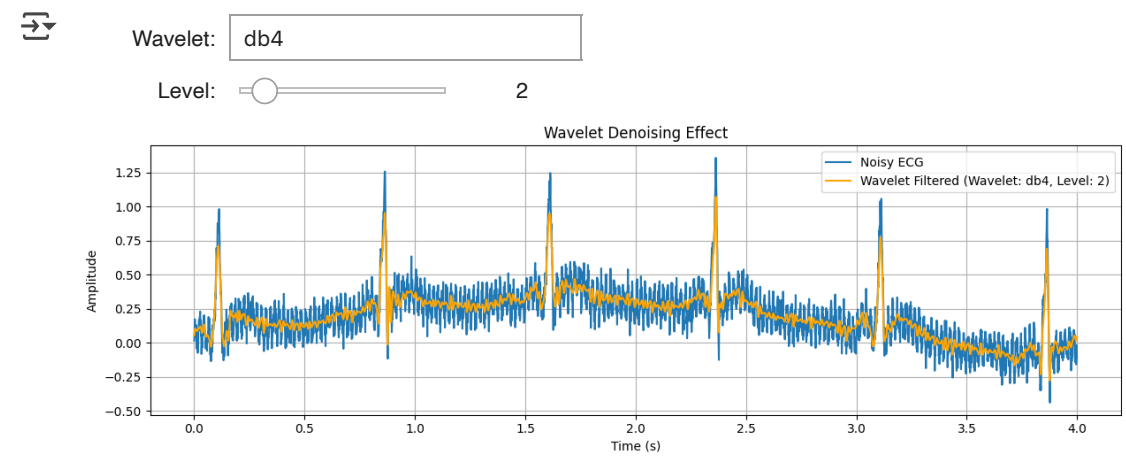




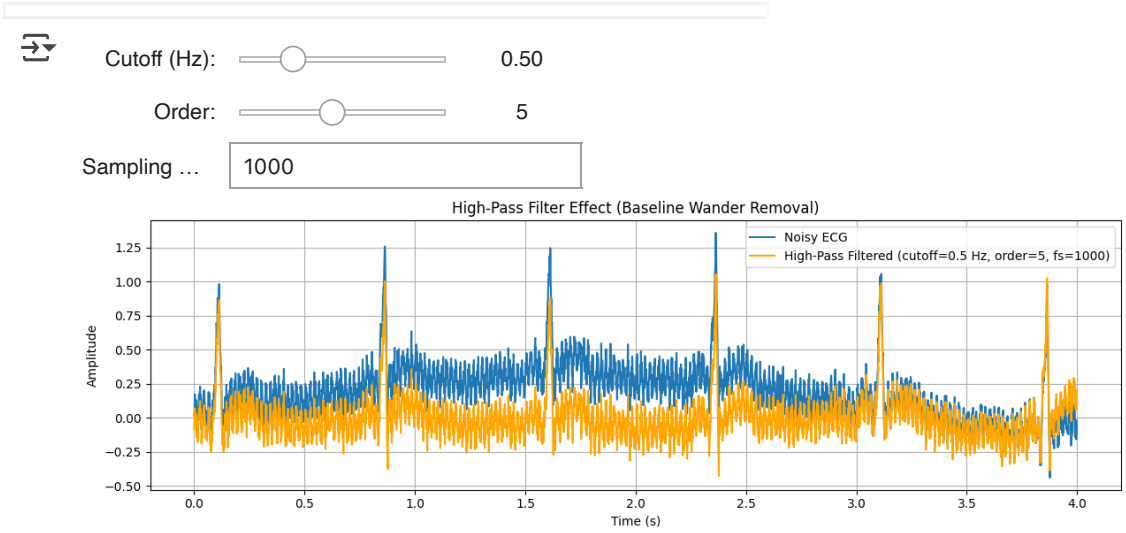

```
# wavelet Filter only
hp_filtered = apply_wavelet_denoising(hp_filtered, wavelet='db4', level=2)
plt.figure(figsize=(12,4))
plt.plot(t, ecg_signal_all, label="Noisy ECG")
plt.plot(t, hp_filtered, label="After wavelet Filter", color='orange')
plt.title("wavelet Effect (High-frequency noise/Gaussian noise Removal)")
plt.legend()
plt.grid(True)
plt.show()
```



```
import matplotlib.pyplot as plt
import numpy as np
import ipywidgets as widgets
from IPython.display import display
from scipy.signal import butter, filtfilt

# Assuming you already have these:
# ecg_signal_all = your noisy signal
# t = time vector
# apply_wavelet_denoising() function already defined

# Create widgets
wavelet_dropdown = widgets.Dropdown(
    options=[f'db{i}' for i in range(1, 11)],
    value='db4',
    description='Wavelet:'
)

level_slider = widgets.IntSlider(
    value=2,
    min=1,
    max=10,
    step=1,
    description='Level:',
    continuous_update=False
)

# Define the interactive update function
def update_wavelet_filter(wavelet, level):
    filtered = apply_wavelet_denoising(ecg_signal_all, wavelet=wavelet, level=l

    plt.figure(figsize=(12, 4))
    plt.plot(t, ecg_signal_all, label="Noisy ECG")
    plt.plot(t, filtered, label=f"Wavelet Filtered (Wavelet: {wavelet}, Level:
    plt.title("Wavelet Denoising Effect")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Display interactive widgets
interactive_plot = widgets.interactive(update_wavelet_filter, wavelet=wavelet_c
display(interactive_plot)
```

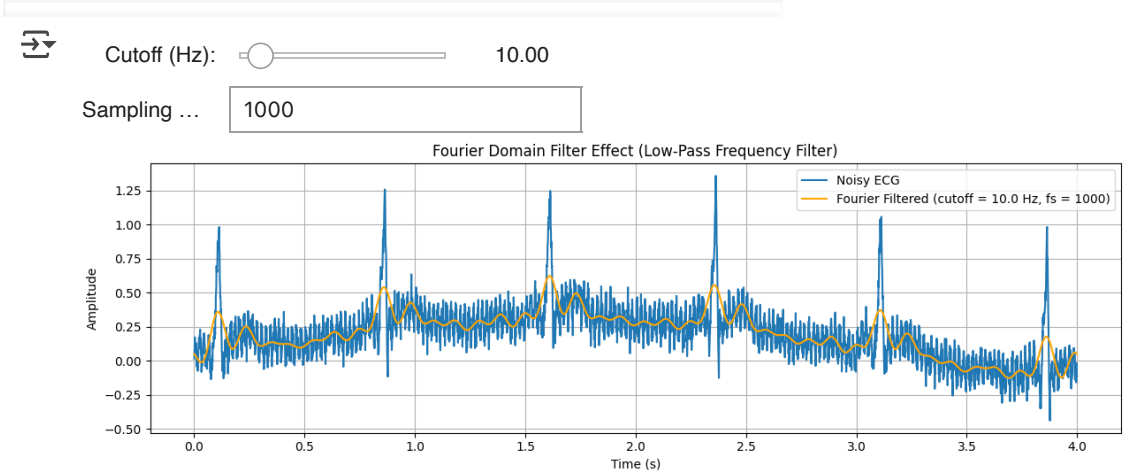


```
from ipywidgets import FloatSlider, IntSlider, Dropdown, interactive
import matplotlib.pyplot as plt
from IPython.display import display

# Widgets
cutoff_slider_hp = FloatSlider(value=0.5, min=0.0, max=2.0, step=0.1, descripti
order_slider_hp = IntSlider(value=5, min=1, max=10, step=1, description='Order:
fs_dropdown_hp = Dropdown(options=[500, 1000, 1500], value=1000, description='S

# Function
def plot_highpass(cutoff, order, fs):
    filtered = apply_highpass_filter(ecg_signal_all, cutoff=cutoff, fs=fs, orde
    plt.figure(figsize=(12, 4))
    plt.plot(t, ecg_signal_all, label="Noisy ECG")
    plt.plot(t, filtered, label=f"High-Pass Filtered (cutoff={cutoff} Hz, order
    plt.title("High-Pass Filter Effect (Baseline Wander Removal)")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

display(interactive(plot_highpass, cutoff=cutoff_slider_hp, order=order_slider_
```




```
import numpy as np
import matplotlib.pyplot as plt

import ipywidgets as widgets
from IPython.display import display

# Define interactive widgets
cutoff_slider_fft = widgets.FloatSlider(value=10, min=1, max=100, step=1, descr
```

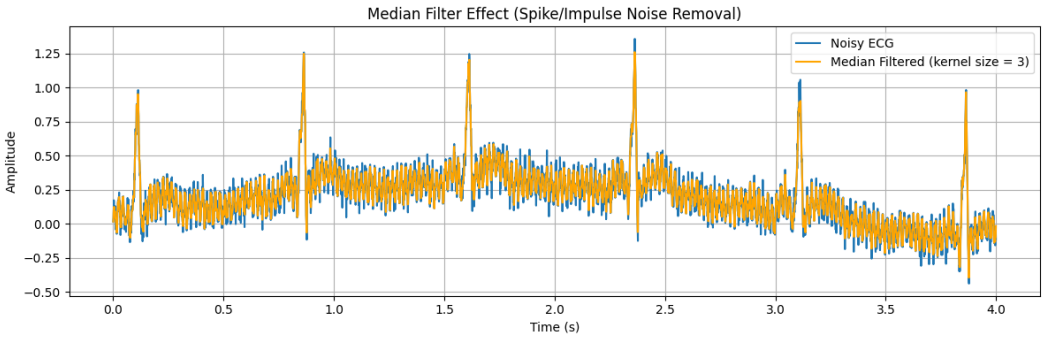
```
fs_dropdown_fft = widgets.Dropdown(options=[500, 1000, 1500], value=1000, descr

# Update function
def plot_fourier_filter(cutoff, sampling_rate):
    # Recalculate FFT using the selected sampling rate
    fft_data = np.fft.fft(ecg_signal_all)
    freqs = np.fft.fftfreq(len(ecg_signal_all), d=1/sampling_rate)
    fft_data[np.abs(freqs) > cutoff] = 0
    filtered = np.fft.ifft(fft_data).real

    plt.figure(figsize=(12, 4))
    plt.plot(t, ecg_signal_all, label="Noisy ECG")
    plt.plot(t, filtered, label=f"Fourier Filtered (cutoff = {cutoff} Hz, fs =
    plt.title("Fourier Domain Filter Effect (Low-Pass Frequency Filter)")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Display the interactive interface
display(widgets.interactive(plot_fourier_filter, cutoff=cutoff_slider_fft, samp
```




```
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display
from scipy.signal import medfilt

# Median filter function
def apply_median_filter(data, kernel_size=3):
    return medfilt(data, kernel_size)

# Widget for kernel size
kernel_slider = widgets.IntSlider(
    value=3, min=1, max=21, step=2, description='Kernel Size:', continuous_upda
)

# Plotting function
def plot_median_filter(kernel_size):
    filtered = apply_median_filter(ecg_signal_all, kernel_size=kernel_size)

    plt.figure(figsize=(12, 4))
    plt.plot(t, ecg_signal_all, label="Noisy ECG")
```

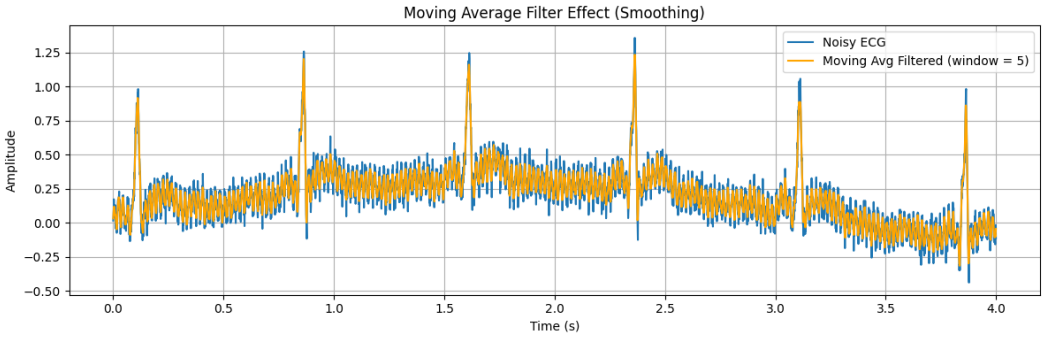
```
    plt.plot(t, filtered, label=f"Median Filtered (kernel size = {kernel_size})
    plt.title("Median Filter Effect (Spike/Impulse Noise Removal)")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Display widget
display(widgets.interactive(plot_median_filter, kernel_size=kernel_slider))
```

Kernel Size: 3



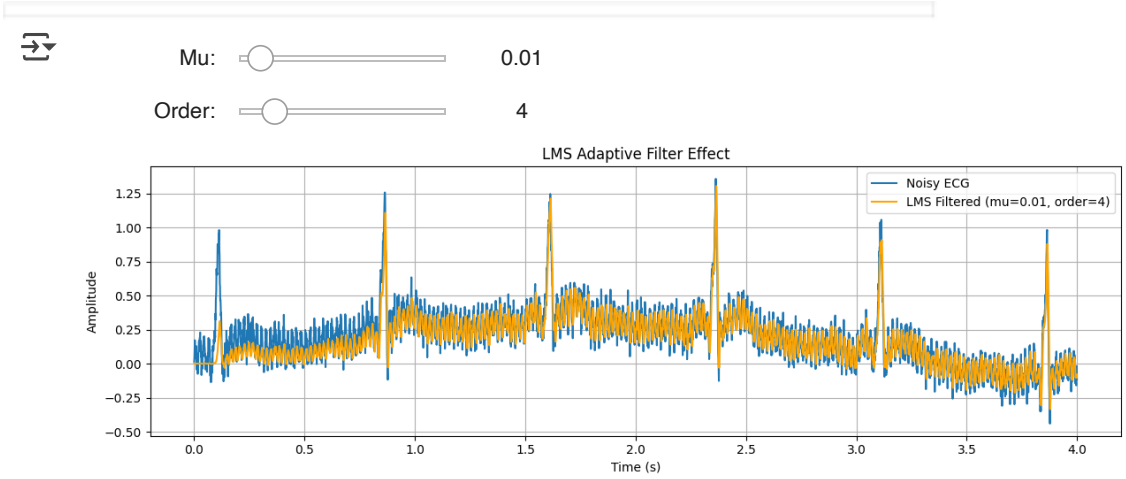

```
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display

# Function to apply a moving average filter
def apply_moving_average_filter(data, window_size=5):
    if window_size < 1:
        raise ValueError("Window size must be at least 1.")
    return np.convolve(data, np.ones(window_size) / window_size, mode='same')

# Create widget for window size
window_slider_ma = widgets.IntSlider(
    value=5, min=1, max=50, step=1, description='Window Size:', continuous_upda
)

# Define interactive plotting function
def plot_moving_average(window_size):
    filtered = apply_moving_average_filter(ecg_signal_all, window_size=window_s

    plt.figure(figsize=(12, 4))
    plt.plot(t, ecg_signal_all, label="Noisy ECG")
    plt.plot(t, filtered, label=f"Moving Avg Filtered (window = {window_size})'
    plt.title("Moving Average Filter Effect (Smoothing)")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Display interactive widget
display(widgets.interactive(plot_moving_average, window_size=window_slider_ma))
```

Window Size: 5




```
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display

# LMS filter function
def lms_filter(input_signal, mu, order):
    n = len(input_signal)
    y = np.zeros(n)
    e = np.zeros(n)
    w = np.zeros(order)
    for i in range(order, n):
        x = input_signal[i-order:i][::-1]
        y[i] = np.dot(w, x)
        e[i] = input_signal[i] - y[i]
        w += mu * e[i] * x
    return y

# Widgets
mu_slider = widgets.FloatSlider(value=0.01, min=0.001, max=0.1, step=0.001, des
order_slider = widgets.IntSlider(value=4, min=1, max=20, step=1, description='O

# Plotting function
def plot_lms_filter(mu, order):
    filtered = lms_filter(ecg_signal_all, mu=mu, order=order)

    plt.figure(figsize=(12, 4))
    plt.plot(t, ecg_signal_all, label="Noisy ECG")
    plt.plot(t, filtered, label=f"LMS Filtered (mu={mu}, order={order})", colo
```

```
    plt.title("LMS Adaptive Filter Effect")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Display widget
display(widgets.interactive(plot_lms_filter, mu=mu_slider, order=order_slider))
```



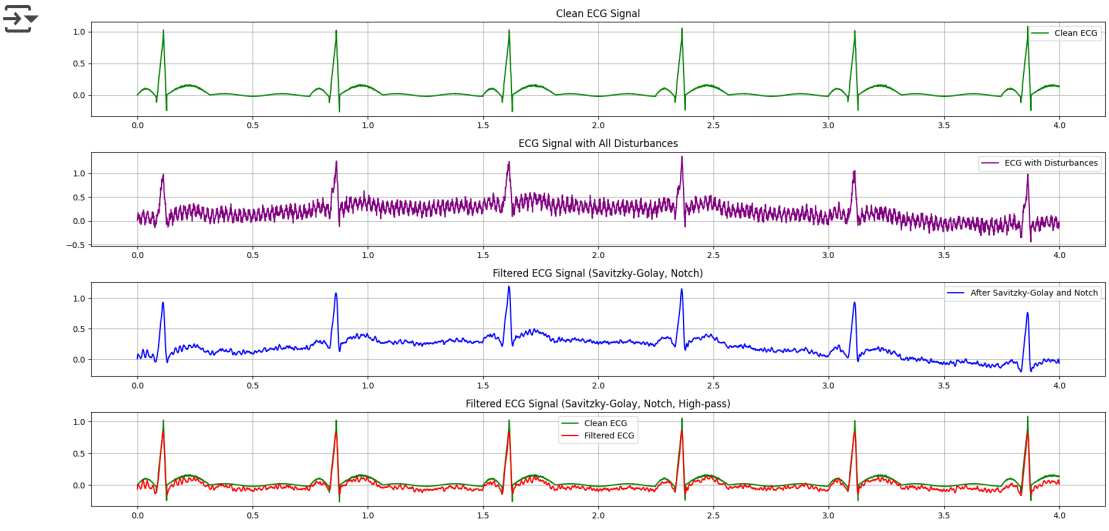
Filter Type	Best For	Key Parameters
High-Pass Filter	Baseline Wander	cutoff = 0.5 Hz, order = 4–6
Notch Filter	50 Hz Powerline Interference	cutoff = 50 Hz, Q = 30, fs = 1000 Hz
Savitzky-Golay	Gaussian Noise (Smooths)	window = 31, polyorder = 3–7
Wavelet Denoising	Gaussian Noise (Preserves peaks)	wavelet = db4, level = 2–3
Median Filter	Impulse/Spike Noise	kernel size = 3–7
Moving Average	General Smoothing	window size = 5–15
LMS Adaptive Filter	Unknown or non-stationary noise	mu = 0.01, order = 4–10
Fourier Filter	General low-pass filtering	cutoff = 10–30 Hz, fs = 1000 Hz

By observing the behavior of each filter under different parameter settings, we evaluated their effectiveness in removing specific types of noise while preserving the original ECG waveform.

- The High-pass filter (cutoff ≈ 0.5 Hz) effectively eliminated baseline wander, which typically arises due to patient motion or breathing.
- The Notch filter (centered at 50 Hz with Q ≈ 30) accurately suppressed powerline interference without distorting the signal.
- The Savitzky-Golay filter (window length 31, polynomial order 3–7) smoothed out

Gaussian high-frequency noise while maintaining the sharpness of important ECG components like the QRS complex.

```
# Step 1: Savitzky-Golay first
filter1 = savgol_filter(ecg_signal_all, window_length=31, polyorder=7)

# Step 2: Notch second
filter2 = apply_notch_filter(filter1, cutoff=50, fs=1000, quality_factor=30)

# Step 3: High-pass last
filter3 = apply_highpass_filter(filter2, cutoff=0.5, fs=1000, order=4)

# Plot & metrics

plt.figure(figsize=(18, 9))
plt.subplot(4, 1, 1)
plt.plot(t, ecg_signal, label='Clean ECG', color='green')
plt.title('Clean ECG Signal')
plt.legend()
plt.grid(True)

plt.subplot(4, 1, 2)
plt.plot(t, ecg_signal_all, label='ECG with Disturbances', color='purple')
plt.title('ECG Signal with All Disturbances')
plt.legend()
plt.grid(True)

plt.subplot(4, 1, 3)
plt.plot(t, filter2, label='After Savitzky-Golay and Notch', color='blue')
plt.title('Filtered ECG Signal (Savitzky-Golay, Notch)')
plt.legend()
plt.grid(True)

plt.subplot(4, 1, 4)
plt.plot(t, ecg_signal, label='Clean ECG', color='green')
plt.plot(t, filter3, label='Filtered ECG', color='red')
plt.title('Filtered ECG Signal (Savitzky-Golay, Notch, High-pass)')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

mse = mean_squared_error(ecg_signal, filter3)
prd_value = prd(ecg_signal, filter3)
print(f"Order: Savitzky-Golay → Notch → High-pass")
print(f"MSE: {mse:.5f}, PRD: {prd_value:.2f}%")
```



```
Order: Savitzky-Golay → Notch → High-pass
MSE: 0.00406, PRD: 43.26%
```

ECG Signal Denoising Summary

To eliminate various types of noise from the ECG signal, we applied a sequence of three filters. Each filter targets a specific type of disturbance and collectively enhances the clarity and diagnostic quality of the ECG waveform. In this study, we focused on exploring different filter orders using the same set of filters to determine which sequence most effectively reduces noise while preserving the original signal characteristics.

The aim of experimenting with different filter combinations is to remove noise and restore the ECG signal as closely as possible to its original form. Studying various filters and their characteristics, we selected five filter combinations that are likely to reduce noise effectively

as shown in the above simulation. Mean Squared Error (MSE) and Percent Root Difference (PRD) shows how filters is close the orginal signal.

- Mean Squared Error (MSE): MSE measures the average of the squares of the differences between the original clean signal and the filtered (denoised) signal.
- Percent Root Differenc(PRD): PRD is a normalized error metric that expresses the difference between the original and filtered signals as a percentage of the original signal's energy.

Lower values of MSE and PRD means filtered signal is close to orginal signal. That means, from the above expriment we know that High-Pass + Notch + Savitzky-Golay is giving less MSE and PRD values.

Order of the filters

After testing various filter order combinations, the results are summarized below. The best-performing combination in terms of noise reduction and signal preservation is:

Savitzky-Golay → Notch → High-pass

This sequence effectively removes smooths high-frequency noise, powerline interference, and eliminates baseline wander.

Filter Order	MSE	PRD
1. Savitzky-Golay → High-pass → Notch	0.00476	46.88%
2. Notch → Savitzky-Golay → High-pass	0.00406	43.27%
3. High-pass → Savitzky-Golay → Notch	0.00489	47.48%
4. Savitzky-Golay → Notch → High-pass	0.00406	43.26%
5. Notch → High-pass → Savitzky-Golay	0.00413	43.66%

Filter Application Order

1. **Savitzky-Golay Filter**
    - Firstly, to smooth residual high-frequency noise(Gaussian noise) without affecting key signal features.

2. **Notch Filter (50 Hz)**
    - Second, It clean up powerline interference and make the signal ready for further filtering.
3. **High-pass Filter**
    - Last, Highpass filter removes baseline wander.

This step-by-step approach ensures that the ECG signal is cleaned in a structured, non-destructive way, preserving clinical features while eliminating unwanted noise.